

Oblig 3 - ARM assemblerprogrammering

Assemblerprogrammering

Assembler er den laveste (med noen unntak) måten å programmere en CPU. Programmeringsspråket er en menneskeleselig versjon av instruksjonene som maskinvaren utfører. Sett av instruksjoner som kan utføres er forskjellig avhengig av prosessorarkitektur, og assemblerkode er derfor spesifikt til prosessoren det kjøres på. Til sammenligning må høyere nivå språk, som C eller Python, oversettes til maskininstruksjoner. Dette gjøres enten av en kompilator eller ved at koden tolkes under kjøring. I denne obligen skal vi jobbe med ARM Assembler som er beskrevet i kapittel 6.3 i læreboken.

Assemblerprogrammering er noe man sjeldent kommer borti i dagliglivet som programmerer. Allikevel finnes det gode grunner til å kunne skrive og lese Assembler. For mange mikroprosessorer trenger man assemblerprogrammering for å kunne utnytte alle egenskapene til prosessoren. Assembler brukes også blandt annet dersom man skal skrive kompilatorer, drivere eller operativsystemer. I denne obligen skal vi implementere noen enkle algoritmer i Assembler. Dette vil gi en innføring i hvordan enkle byggeblokker kan kombineres for å skape funksjonalitet.

Denne obligen består av fem deler. I første del av obligen skal vi implementere en algoritme som regner ut Fibonacci sekvensen i Assembler. I andre del skal vi utvide denne koden ved å gjøre den om til en funksjon vi kan kalle. I tredje del skal vi se hvordan en kompilator oversetter fra et høyere nivå språk(C) til lav nivå Assembler. I del fire og fem skal vi se hvordan moderne datamaskiner representerer flyttall og hvordan man kan legge sammen slike tall. Selv om flyttall er støttet i prosessoren vil det å implementere det selv være en fin øvelse for å jobbe med binærtall og for å lære mer om lavnivå programmering.

I den siste delen av obligen vil det være en liten konkurranse. Vinneren av konkurransen vil få en premie i siste forelesning.

Oppgaven skal leveres som en PDF-fil med svar på tekstoppagvene. For programmeringsoppagvene skal kildekoden legges ved som egne filer.

Del 1:

Det første vi skal implementere i denne obligen er en algoritme for å beregne det N-te Fibonacci tallet. Vi kommer til å begrense oss til positive heltall og null for N. Nedenfor har vi gjengitt de fem første tallene i rekken.

N = 0	N = 1	N = 2	N = 3	N = 4
0	1	1	2	3

Denne rekken er beskrevet av rekurrensen $F(N) = F(N - 1) + F(N - 2)$ med grunnstegene $F(0) = 0$ og $F(1) = 1$. Algoritmen er vist i pseudokode under

```
current = 0
previous = 1
n = 13
while n > 0:
    temporary = current
    current = previous + current
    previous = temporary
    n = n - 1
```

```
# `current` contains the answer!
```

For å komme i gang med assemblerprogrameringen raskere benytter vi kodeskjelettet under.

```
.text
.global main

@ Main function of program
main:
    @ First load the desired Fibonacci number
    MOV r0, #13 @ N
    @ Initialize the first two numbers in the sequence
    MOV r1, #0 @ current
    MOV r2, #1 @ previous

@ Loop will do the main work calculating the Fibonacci sequence
loop:
    @ ! Implement your logic here !
    @ Jump to start off loop
    B loop
@ To be a good citizen we branch (and exchange) on the lr register
@ to return to the caller
exit:
    BX lr
```

I kodeskjelettet har vi gjort følgende for å hjelpe deg med å komme i gang:

- Vi plasserer verdien 13 i register 0 (r0), dette indikerer at vi ønsker å beregne Fibonacci tallet for $N = 13$.
- Vi har så initialisert to registre(r1 og r2) med de to første tallene i Fibonacci rekken.

Oppgaven er å fylle ut algoritmen og se til at resultatet plasseres i register 0 (r0). Husk at du kan endre initialiseringen av r0 for å teste forskjellige verdier av N (test med flere verdier for å forsikre deg om at oppgaven er løst riktig).

Kompilere koden

Lagre koden ovenfor i en fil med navnet part1.s. For å kunne compilere og kjøre koden må vi benytte oss av en kompiletor, som kan gjøre om kildekoden til en kjørbart fil. Som du kanskje husker fra Oblig 1 må vi bruke emulatormiljøet (som aktiveres med *in2060_arm* på ifi pc'er), eller en maskin med ARM arkitektur, for å compilere og kjøre koden. Den følgende kommandoen vil compilere kildekoden:

```
I Emulatormiljøet:
arm-linux-gnueabi-g++ -static -g -o part1 part1.s

På RPi:
gcc -g -o part1 part1.s
```

Kjør programmet ved hjelp av gdb-tui med følgende kommandoer:

```
I Emulatormiljøet:
qemu-arm-static -g 1234 ./part1 &
gdb-multiarch -tui ./part1
target remote localhost:1234
tui reg general

På RPi:
gdb -tui ./part1
tui reg general
```

En oversikt over kommandoer du kan bruke i gdb-tui finnes for eksempel [her](#).

Når du er ferdig med oppgaven (og underveis) kjør programmet med gdb-tui, og se til at du kan se riktig verdi liggende i r0 i slutten av programmet (Husk: `tui reg general` for å vise innholdet i registerene).

Innlevering:

- **Kildekode til Fibonacci algorithmen i Assembler (part1.s)**

Del 2:

En viktig inndeling i mange programmeringsspråk er *funksjoner*. Funksjoner kan brukes til å lage mindre blokker med separert logikk. Nå skal vi se hvordan dette fungerer på det laveste nivået.

I denne oppgaven skal vi utvide Fibonacci algoritmen fra forrige del til å bli en funksjon vi kan kalle. Når man skriver en funksjon i assembler er det viktig å følge funksjonskallkonvensjonen (calling convention på engelsk), som er beskrevet i kapittel 6.3.7 i læreboka. Denne konvensjonen beskriver hvilke registre som benyttes til inngangsverdier og returverdier, og hvilke registre funksjoner har lov til å endre på. Det er viktig å passe på at alle registre man ikke har lov til å endre har samme verdi når man returnerer fra funksjonen som det de hadde da funksjonen ble kallt. Hvis man ikke passer på dette kan funksjonskalleren få seg en overraskelse når tallene som var der har forsvunnet.

Lag en ny kildekode fil med navnet `part2.s`, fyll inn skjelettet under og implementer funksjonen som heter `fib`. Funksjonen skal ta inn et argument, det ønskede Fibonacci tallet `N`, og skal returnere et tall, det beregnede svaret. I `main` skal `fib` funksjonen kalles, deretter skal resultatet skrives ut med `printf`.

`printf` kan kalles på som alle andre funksjoner i Assembler (dvs. `printf` kalles på akkurat samme måte som `fib`). For våre formål vil vi behandle den som en funksjon som tar inn to argumenter, vår formateringsstreng og tallet vi ønsker å skrive ut.

```
.text
.global main
@ Function that `main` should call
fib:
    @ Fill in the Fibonacci algorithm here
    @ ensure that input arguments and return values
    @ follows the ARM calling convention, section `6.3.7`
    @ Also note that if you need extra registers it might
    @ be necessary to store previous state on the stack
    BX lr

main:
    @ Always return properly to caller (even from main)
    BX lr

@ The 'data' section contains static data for our program
.data
output_string:
    .asciz "%d\n"
```

Formateringsstrengen kan lastes inn ved hjelp av `LDR` og `=output_string`.

Det finnes to måter å returnere fra et funksjonskall på. Vi vil i motsetning til boken anbefale at du bruker `BX` for å returnere fra et funksjonskall, men begge metoder godtas under innlevering.

Kompiler og bruk gdb-tui for å teste programmet ditt som i forrige deloppgave. Når du er ferdig med oppgaven prøv å kjøre programmet uten gdb-tui ved å kalle på programmet ditt i terminalen (Med emulator: `qemu-arm-static part2` Med RPi: `./part2`).

Tekstoppgaver

1. Når man kaller en funksjon i Assembler er det noen registre kalleren av funksjonen må passe på å lagre unna fordi funksjonen kan endre verdien i registerene. Andre registre skal ha samme verdi etter et kall som de hadde før kallet. Under ARM funksjonskallkonvensjonen, hvilke registre er det kalleren av funksjonen som har ansvaret for å ta vare på verdiene i, og hvilke registre er det funksjonen som har ansvaret for?
2. Hva skjer med input argumentet til fib funksjonen etter at fib returnerer? (Hint: Tenk på hva som ligger i registeret der input argumentet lå tidligere)
3. Hvilken endring måtte du gjøre i *main* fra Del 1 slik at programmet avsluttes riktig, og hvorfor måtte dette gjøres?

Innlevering:

- Kildekode med oppdatert Fibonacci algorithm i Assembler (part2.s)
- Svar på tekstoppgaver (pdf)

Del 3:

Det er kanskje ikke så mange tilfeller hvor man får bruk for å skrive ren Assembler kode i hverdagen, men det er viktig å kunne forstå Assembler koden som høyere nivå programmeringsspråk produserer. Dette kan være for å finne feil eller for å oppdage bedre måter å utnytte CPU ressursene på. I denne oppgaven skal vi se på hvordan kompilatoren GCC oversetter C-kode til Assembler instruksjoner.

Under har vi laget et C program for å regne ut det N-te Fibonacci tallet.

```
#include <stdio.h>

// Our now familiar Fibonacci function
int fib(unsigned int n) {
    // Initialize variables
    int curr = 0;
    int prev = 1;
    // Loop until done
    while(n != 0) {
        int tmp = curr;
        curr = curr + prev;
        prev = tmp;
        n = n - 1;
    }
    // Return as usual
    return curr;
}

// Main is required in C and "should" return an integer,
// for now we can ignore the arguments to this function
int main(int argc, char** argv) {
    // Calculate the 13th Fibonacci number
    const int f = fib(13);
    printf("%d\n", f);
    // In C main should return a "status" of the program when
    // exiting, '0' means successful (program terminated as
    // expected)
    return 0;
}
```

Lagre dette i en fil med navnet part3.c.

Tekstoppgaver

For å undersøke hva kompilatoren gjør med kildekoden må vi stoppe kompilatoren før den gjør om kildekoden til maskinkode. Dette kan gjøres ved å sende inn et flag, -S. Kjør følgende kommando og undersøk den produserte Assembler koden for å svare på spørsmålene under.

```
arm-linux-gnueabi-g++ -O0 -S -o part3.s part3.c
```

Legg merke til at vi har fjernet `-g` og lagt til `-O0`. For denne oppgaven så trenger vi ikke avluserinformasjon (debug information) som kan lage mye støy i den produserte Assembler koden. `-O0` forteller kompilatoren at vi ønsker å optimere for størrelse noe som burde gi en Assembler kode som er enklere å tolke.

1. Undersøk den produserte Assembler koden og sammenlign med din egen kildekode fra Del 2, er det noen store forskjeller mellom det du lagde og det GCC produserer?
2. Endre `-O0` til `-O2`, hvordan påvirker dette den produserte koden? Prøv å endre til `-O3`, ser du noen forandring nå?
3. Hvilke argumenter er det for og i mot å bruke en kompilator sammenlignet med å skrive Assembler? Tenk spesielt på hvor mye jobb det ville være å oversette programsnittene i Del 1, 2 og 3 til en annen prosessorarkitektur.

Innlevering:

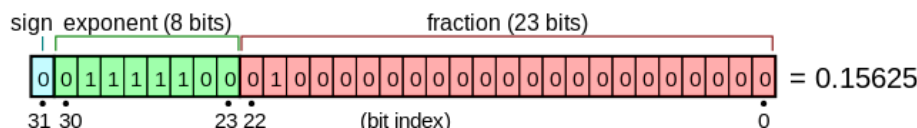
- Svar på tekstopp-gaver (i samme pdf som tidligere tekstopp-gaver)

Flyttall (IEEE 754)

Flyttall, mer spesifikt IEEE 754 32-biter flyttall (beskrevet i kapittel 5.3.2), er en måte å representere desimaltall i binær på, slik at man kan regne med dem på en datamaskin. IEEE 754 er en standard som så godt som alle prosessorarkitekturer støtter. Standarden beskriver hvordan flyttall skal operere, og gjør at det er forutsigbart hva som skjer når vi jobber med desimaltall på forskjellige maskiner.

Når vi lagrer desimaltall som flyttall på en datamaskin mistes noe informasjon grunnet avrunding. I tallsystemet kan tallet $1/10$ representeres nøyaktig som 0.1 . På grunn av representasjonen vi bruker for flyttall kan ikke dette tallet representeres nøyaktig i datamaskinen, det nærmeste vi kommer er 0.10000000000000001 . Dersom vi gjør mange flyttallsoperasjoner kan disse små avrundingsfeilene etter hvert "balle på seg", og svaret vårt kan bli feil. Det er derfor viktig at man kjenner til styrkene og svakhetene til flyttall slik at man ikke ender opp med feil.

I bildet under kan du se hvordan IEEE 754 32-biter flyttall er representert.



IEEE 754 flyttall

Et flyttall består av et fortegn (sign på bildet), eksponent (exponent) og en desimal (fraction). Tallet representerer desimaltall ved hjelp av vitenskapelig notasjon, men isteden for å bruke 10 som grunntall brukes 2.

For å representere 228 som flyttall (ikke IEEE 754) konverterer vi først tallet til binær, $228 = 11100100$, deretter konverterer vi til vitenskapelig notasjon $11100100 = 1.11001 \times 2^7$. Tallet 111001 kan så plasseres i desimalen, mens 7 kan plasseres i eksponenten. IEEE 754 går litt lengre for å få litt mer ut av bitene, men det er best forklart i boken (legg merke til biasert eksponent og utnyttelse av desimalen).

Del 4:

Før vi implementerer en algoritme for å legge sammen flyttall trenger vi litt oppvarming i flyttallsregning. Gjør følgende oppgaver for hånd og vis fremgangsmåten din.

1. Oversett 2.0 til IEEE 754 32-biter flyttall.
2. Oversett 3.0 til IEEE 754 32-biter flyttall.
3. Oversett 0.50390625 til IEEE 754 32-biter flyttall.
4. Legg sammen tallene 2.0 og 0.50390625 med samme fremgangsmåte som Figur 5.30 i boka. (du trenger **ikke** å ta hensyn til avrunding).

Innlevering:

- Svar på tekstoppgaver (i samme pdf som tidligere tekstoppgaver)

Del 5:

Til slutt skal vi implementere addisjon av flyttall i Assembler uten bruk av innebygde flyttallsoperasjoner. Vi anbefaler at du leser deg opp på assembleroperasjonene AND/ORR, LSR/LSL og BIC beskrevet i kapittel 6.3.1 og 6.3.2.

For å gjøre oppgaven litt enklere kommer vi til å anta at det bare er positive flyttall som skal adderes. Koden trenger heller ikke å ta hensyn til avrunding. Du trenger ikke å lage en egen funksjon for addisjonen, men kan utføre hele beregningen i hovedfunksjonen.

For å komme i gang kan du bruke skjelettet under og lagre det som part5.s.

```
@ The two numbers we want to add
num1:  .word  0x3f800000
num2:  .word  0x3f800000

.text
.global main
main:
    @ Load numbers
    LDR r0, num1
    LDR r1, num2
    @ ! Perform addition here !
    @ When done, return
    BX lr
```

Algoritmen

Flyttallsaddisjonen dere skal utføre er beskrevet i kap. 5.3 i læreboka. Oppsummert skal koden din gjøre følgende. (Det er ikke lov til å bruke noen av flyttallsinstruksjonene som finnes i ARM Assembler.)

1. Maskere ut og flytte ned eksponenten i begge tall, slik at eksponenten ligger i de minst signifikante bit'ene i registeret.
2. Maskere ut desimalen, og legge til et ledende 1 tall.
3. Trekke den minste eksponenten fra den største og sette eksponenten til det nye tallet lik den største av de to eksponentene.
4. Høyre skifte desimalen til det minste tallet med forskjellen av eksponentene fra steget over.
5. Summere desimalene.
6. Normalisere resultatet hvis nødvendig. (høyre skift desimalen og øk eksponenten med 1)
7. Fjerne ledende 1 fra den nye desimalen og konstruere det nye flyttallet med fortegn, eksponent og desimal.
8. Legg resultatet i r0.

Test algoritmen din med følgende addisjoner for å overbevise deg selv om at alt er riktig. Lagre resultatet av addisjonene i en tekstfil med navn addisjoner.txt.

1. $1.0 + 1.0 = 2.0$ ($0x3F800000 + 0x3F800000 = 0x40000000$)
2. $2.0 + 1.0$
3. $3.0 + 3.5$
4. $2.0 + 0.50390625$

Som i tidligere deloppgaver kompiler og test med gdb-tui.

Innlevering:

- Kildekode til flyttallsaddisjon i Assembler (part5.s)
- Teksfilen addisjoner.txt

Lever alle filene i canvas i en zippet mappe. (Filer som skal med: pdf med tekstoppgaver, part1.s, part2.s, part5.s, addisjoner.txt)

Konkurranse:

Når du er ferdig med obligen kan du velge å jobbe litt videre med assembler, og delta i denne konkurransen. (Vi anbefaler at du gjør deg helt ferdig med obligen før du begynner på denne delen.) Konkurransen består av å ta programmet du laget i del 5, og gjøre det så kort som overhodet mulig. Her gjelder det altså å være smart og velge effektive instruksjoner slik at programmet har så få linjer som mulig. Programmet med færrest linjer vinner konkurransen! Vinneren får en liten premie i siste forelesning. (Husk: Å korte ned programmet til færrest mulig linjer er ikke alltid lurt, ettersom det ofte gjør koden mindre forståelig. Det kan allikevel være en morsom øvelse for å få mer greie på hvilke instruksjoner som finnes i assembler, og hvordan man kan bruke dem effektivt.)

Regler:

- Tomme linjer eller linjer som kun inneholder kommentarer teller ikke med på linjeantallet
- Det er ikke lov til å bruke noen av flyttallsinstruksjonene som finnes i ARM Assembler
- Det er ikke lov til å kalle eksterne funksjoner
- Bidraget til konkurransen leveres i en egen fil med navn konkurranse.s

Husk å kopiere løsningen fra Del 5 av obligen over i en ny fil før du begynner å endre på den, slik at du ikke risikerer å introdusere feil i løsningen din på obligen.